



भारतीय सूचना प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Information Technology Guwahati

CS236 - ARTIFICIAL INTELLIGENCE LAB
ASSIGNMENT - 07

Problem Statement

Write a Python program to solve the **Traveling Salesman Problem (TSP)** using the following optimization algorithms:

1. **Hill Climbing**
2. **Simulated Annealing**

The program must:

1. **Input:** Accept the number of cities n and an $n \times n$ cost matrix from standard input. The matrix is symmetric with a zero diagonal, where $\text{cost}[i][j]$ denotes the travel cost from city i to city j .
2. **Solve:** Implement both Hill Climbing and Simulated Annealing to find approximate solutions to the TSP.
3. **Output:** For each algorithm, print the tour (a sequence of cities starting and ending at city 0) and the total associated cost.

Algorithm Specifications

1. Hill Climbing:

- i. Initialize with a random tour: a permutation of cities 1 to $n - 1$ inserted between two occurrences of city 0 (i.e., city 0 is fixed as the start and end point throughout).
- ii. Iteratively:
 - i. Generate all neighbors by swapping any two distinct *non-zero* cities in the tour (city 0 must remain fixed at both ends).
 - ii. Move to the neighbor with the lowest cost if it strictly improves upon the current tour cost.
 - iii. Stop when no improving neighbor exists (local optimum reached).
- iii. Output the final tour and its total cost.

2. Simulated Annealing:

- i. Initialize with a random tour (same format as above, with city 0 fixed at both ends).
- ii. Set initial temperature $T = 1000$ and cooling rate $\alpha = 0.995$.
- iii. Run for $K = 100 \times n^2$ iterations:
 - i. Randomly select two distinct non-zero cities and swap them to form a neighbor tour.
 - ii. Compute the cost difference $\delta = \text{new_cost} - \text{current_cost}$.
 - iii. Accept the neighbor unconditionally if $\delta < 0$; otherwise accept it with probability $e^{-\delta/T}$.
 - iv. Update the temperature: $T \leftarrow T \times \alpha$.
 - v. Track the best tour (lowest cost) encountered across all iterations.
- iv. Output the best tour found and its total cost.
- v. **Parameter Exploration:** Run the algorithm with at least three distinct parameter combinations and compare results. Suggested combinations are shown in Table 1.

Config	T	α	K	Expected Behavior
1 (default)	1000	0.995	$100 \times n^2$	Balanced exploration and convergence
2	500	0.990	$50 \times n^2$	Faster convergence, may miss global optimum
3	2000	0.900	$200 \times n^2$	Aggressive cooling, wide early exploration

Table 1: Simulated Annealing parameter configurations for exploration.

For each configuration, note the resulting tour cost and comment briefly on solution quality and convergence behaviour.

Input Format

The program reads from standard input:

1. First line: a single integer n (number of cities, $2 \leq n \leq 10$).
2. Next n lines: n space-separated integers each, representing the symmetric cost matrix. The diagonal entries are 0 and $\text{cost}[i][j] = \text{cost}[j][i] > 0$ for $i \neq j$.

Output Format

The program writes to standard output:

1. **Hill Climbing Solution:**
 - i. Route: a sequence of cities, e.g., 0 -> 1 -> 3 -> 2 -> 0
 - ii. Cost: the total tour cost (integer)
2. **Simulated Annealing Solution:**
 - i. Route: a sequence of cities, e.g., 0 -> 2 -> 3 -> 1 -> 0
 - ii. Cost: the total tour cost (integer)

Note: Due to random initialisation and stochastic decisions, actual outputs may differ from the sample below. What matters is that the route is a valid Hamiltonian cycle starting and ending at city 0, and that the reported cost matches the route.

Sample Input

```
4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
```

Sample Output

```
Hill Climbing Solution:
Route: 0 -> 1 -> 3 -> 2 -> 0
Cost: 80
```

```
Simulated Annealing Solution:
Route: 0 -> 1 -> 3 -> 2 -> 0
Cost: 80
```

The optimal tour for the sample input has cost 80. Both algorithms should find this or report a higher cost if they converge to a sub-optimal local minimum.

Submission Requirements

Submit the following:

1. **Source code** implementing the TSP solver (single file, clearly commented).
2. **A short PDF report** containing:
 - i. Two sample inputs with their corresponding outputs.
 - ii. A comparison table showing results for all three Simulated Annealing parameter configurations (tour cost and any observations on convergence).